SUPERIOR UNIVERSITY

Lahore, Pakistan

LAB MANUAL

Artificial Intelligence

(Python)

Name:	Asad
Roll Number:	141
Class:	BSAI 4C
Instructor:	Rasikh Ali
Faculty:	Software Engineering

Faculty of Software Engineering

SUPERIOR UNIVERSITY LAHORE PAKISTAN

List of Experiments

Lab No.	Experiment Name
1	Introduction to Python (& Installation), Syntax, Basic Functions, Control Structures, Loops & Functions
2	Lists, Tuples, Sets & Dictionary, Classes & Inheritance, Modules in Python & Examples
3	Simple Reflex Agents, Model based Reflex Agents
4	Graph in Python
5	Uninformed Search
6	Searching Algorithms: Informed / Heuristic Search
7	Game Search
8	Machine Learning (File Read, Data Preprocessing)
9	Machine Learning (Naïve Bayes, D-Tree, KNN, SVM)
10	Introduction to Deep Learning
11	Data Visualization using Python
12	Precision, Accuracy, Recall
13	Project
14	Project Evaluation

Lab 1 — Part 1: Introduction to Python

Installing Python

Website: `www.python.org`

For Windows (32 bit): `https://www.python.org/ftp/python/3.4.3/python-3.4.3.msi`

For Windows (64 bit): `https://www.python.org/ftp/python/3.4.3/python-3.4.3.amd64.msi`

Installing Jupyter Notebook

`https://www.anaconda.com/`

Installing PyCharm

`https://www.jetbrains.com/pycharm/download/`

Opening IDLE

Go to the start menu, find Python, and run the program labeled **'IDLE'** (Integrated DeveLopment Environment).

Code Example 1 — Hello, World!

```
>>> print("Hello, World!")
```

Learning Python for a C++/C# Programmer

C++ / C#	Python
Comment begins with `//`	`#`
Statement ends with `;`	No semi-colon needed
Blocks of code defined by `{}`	Defined by indentation (usually four spaces)
Indentation is irrelevant	Must be consistent for same block of code
Conditional: `if-else`	`if – elif – else:`
Parentheses for loop condition required	Not required but colon `:` follows condition

Simple Arithmetic in Python

```
>>> 1 / 2
0.5
>>> 2 ** 3      # Exponent operator
8
>>> 17 / 3       # Classic division returns a float
5.666666666666667
>>> 17 // 3      # Floor division
5
>>> 23 % 3       # Modulus operator
2
```

Python Operators

Command	Name	Example	Output
+	Addition	4 + 5	9
-	Subtraction	8 - 5	3
*	Multiplication	4 * 5	20
/	Classic Division	19 / 3	6.3333
%	Modulus	19 % 3	1
**	Exponent	2 ** 4	16
//	Floor Division	19 // 3	6

Comments in Python

```
# I am a comment. I can say whatever I want!
```

Variables

```
print("This program is a demo of variables")
v = 1
print("The value of v is now", v)
v = v + 1
print("v now equals itself plus one, making it worth", v)
v = v * 5
print("There you go, now v equals", v, "and not", v / 5)
```

Strings

```
word1 = "Good"
word2 = "Morning"
word3 = "to you too!"
print(word1, word2)
sentence = word1 + " " + word2 + " " + word3
print(sentence)
```

Relational Operators

Expression	Function
<	less than
<=	less than or equal to
>	greater than
>=	greater than or equal to
!=	not equal to
==	is equal to

Boolean Logic

Python's Boolean operators are `and`, `or`, and `not`. The `and` operator evaluates as **True** if both arguments are True. The `or` operator evaluates as **True** if either argument is True. The `not` operator inverts the result.

Operator Precedence

Operator	Description
()	Parentheses
**	Exponentiation
~ + -	Complement, unary plus and minus
* / % //	Multiply, divide, modulo, floor division
+ -	Addition and subtraction
>> <<	Right and left bitwise shift
&	Bitwise AND
^ |	Bitwise exclusive OR and regular OR
<= < > >=	Comparison Operators
== !=	Equality Operators
= %= /= //= -= += *= **=	Assignment operators
is, is not	Identity operators
in, not in	Membership operators
not, or, and	Logical operators

Conditional Statements

`if` Statement

```
y = 1
if y == 1:
    print("y still equals 1, I was just checking")
```

`if-else` Statement

```
a = 1
if a > 5:
    print("This shouldn't happen.")
else:
    print("This should happen.")
```

`elif` Statement

```
z = 4
if z > 70:
    print("Something is very wrong")
elif z < 7:
    print("This is normal")
```

TASK

Task: Run and Modify

Open IDLE and run the following program. Try different integer values. Play around with indentation and see what happens. Note any errors and the changes needed to remove them.

```
x = input("Please enter an integer: ")
if x < 0:
    x = 0
    print('Negative changed to zero')
elif x == 0:
    print('Zero')
elif x == 1:
    print('Single')
else:
    print('More')
```

Note: `input()` returns a string. Use `a = input("Enter Value: ")` and `print(a)`.

Indexes of String

```
name = "J. Smith"
print(name, "starts with", name[0])
# Output: J. Smith starts with J
```

String Properties

```
name = "Linkin Park"
length = len(name)
big_name = str.upper(name)
print(big_name, "has", length, "characters")
# Output: LINKIN PARK has 11 characters
```

Strings and Numbers

`ord(text)` converts a string into a number. Example: `ord('a')` is 97.
`chr(number)` converts a number into a string. Example: `chr(99)` is "c".

Loops in Python

The `while` loop

```
a = 0
while a < 10:
    a = a + 1
    print(a)
```

Range Function

```
range(5)        # [0, 1, 2, 3, 4]
range(1, 5)      # [1, 2, 3, 4]
range(1, 10, 2)  # [1, 3, 5, 7, 9]
```

The `for` loop

```
for i in range(1, 5):
    print(i)

for i in range(1, 5):
    print(i)
else:
    print('The for loop is over')
```

Functions

```
def greet():           # function definition
    print("Hello")
    print("Good Morning")

greet()                  # function calling

def add_sub(x, y):
    a = x + y
    b = x - y
    return a, b

result1, result2 = add_sub(5, 10)
print(result1, result2)

def multiplybytwo(x):
    return x * 2

a = multiplybytwo(70)    # a = 140
```

Accessing Values in a List

```
names = ["John", "Alice", "Sarah", "George"]
print(names[2])           # Sarah
names.append("Simon")
print(names[-1])          # Simon
names.remove("John")
print(names)
```

LAB 1 TASK — Mini Projects (Any 1)

1. **Dynamic Calculator** — solving expressions like: `1+2×3(4-5÷4)-(3÷5)`
2. **To-Do List** (Dynamic)
3. **Tic Tac Toe** (Dynamic)
4. **Hangman** (Dynamic) with printing hangman

Lab 2 — Lists, Tuples, Sets & Dictionaries

Lists

Lists are compound data types used to group together values. Each value is numbered starting from zero. List items need not all have the same type.

```
cats = ['Tom', 'Snappy', 'Kitty', 'Jessie', 'Chester']
print(cats[2])            # Kitty
cats.append('Oscar')
del cats[1]               # Remove Snappy
```

Functions of Lists

- `list.append(x)` — Add an item to the end
- `list.extend(L)` — Extend the list by appending all items from L
- `list.insert(i, x)` — Insert at given position
- `list.remove(x)` — Remove first item with value x
- `list.pop(i)` — Remove item at position i and return it
- `list.count(x)` — Count occurrences of x
- `list.sort()` — Sort items in place
- `list.reverse()` — Reverse elements in place

Compound Datatype Examples

```
>>> a = ['spam', 'eggs', 100, 1234]
>>> a[1:-1]
['eggs', 100]
>>> a[:2] + ['bacon', 2*2]
['spam', 'eggs', 'bacon', 4]
>>> a[2] = a[2] + 23
>>> a
['spam', 'eggs', 123, 1234]
>>> a[0:2] = [1, 12]
>>> a
[1, 12, 123, 1234]
>>> len(a)
4
```

Nested Lists

```
>>> q = [2, 3]
>>> p = [1, q, 4]
>>> len(p)
3
>>> p[1]
[2, 3]
```

Tuples

Tuples are like lists, but **immutable** (values cannot be changed).

```
months = ('January', 'February', 'March', 'April', 'May', 'June',
          'July', 'August', 'September', 'October', 'November', 'December')
>>> 'December' in months
True
```

Sets

A set is an **unordered collection with no duplicate elements**. Use curly braces `{}` or `set()`.

```
>>> basket = ['apple', 'orange', 'apple', 'pear', 'orange', 'banana']
>>> fruit = set(basket)
>>> fruit
{'banana', 'orange', 'pear', 'apple'}
>>> 'orange' in fruit
True

>>> a = set('abracadabra')
>>> b = set('alacazam')
>>> a - b       # letters in a but not in b
{'r', 'd', 'b'}
>>> a | b       # letters in either a or b
{'a', 'c', 'r', 'd', 'b', 'm', 'z', 'l'}
>>> a & b       # letters in both
{'a', 'c'}
>>> a ^ b       # letters in a or b but not both
{'r', 'd', 'b', 'm', 'z', 'l'}
```

Dictionaries

Dictionaries store **key-value pairs**. Keys act as an index.

```
phonebook = {'ali': 8806336, 'omer': 6784346, 'shoaib': 7658344, 'saad': 1122345}
phonebook['waqas'] = 1234567
print(phonebook)
del phonebook['shoaib']
print(phonebook)

# Check keys
list(phonebook.keys())
sorted(phonebook.keys())
print('waqas' in phonebook)        # True
print('Emily Everett' in phonebook) # False
```

Exercises

1. Create a `movies` list containing a single tuple with title, director, year, budget.
2. Use `input()` to gather info about another movie.
3. Create a new tuple from gathered values.
4. Use an f-string to print movie name and release year.
5. Add the new movie tuple to the movies collection using `append`.
6. Print both movies.
7. Remove the first movie using any method.

```
# Solution example:
movies = [("The Room", "Tommy Wiseau", "2003", "$6,000,000")]
title = input("Title: ")
director = input("Director: ")
year = input("Year of release: ")
budget = input("Budget: ")
new_movie = title, director, year, budget
print(f"{new_movie[0]} ({new_movie[2]})")
movies.append(new_movie)
print(movies[0])
print(movies[1])
del movies[0]
```

Lab 2 Task — Mini Projects

MiniProject 1: Fizz Buzz

Print the first 100 rounds of Fizz Buzz:

- If divisible by 3 → "Fizz"
- If divisible by 5 → "Buzz"
- If divisible by both 3 and 5 → "Fizz Buzz"
- Otherwise print the number

Use loops, `range`, conditionals, and the modulo operator `%`.

MiniProject 2: Movie Budget Analysis

Given a list of movie tuples (name, budget), calculate the average budget and print movies exceeding it.

```
movies = [
    ("Eternal Sunshine of the Spotless Mind", 20000000),
    ("Memento", 9000000),
    ("Requiem for a Dream", 4500000),
    ("Pirates of the Caribbean: On Stranger Tides", 379000000),
    ("Avengers: Age of Ultron", 365000000),
    ("Avengers: Endgame", 356000000),
    ("Incredibles 2", 200000000)
]
```

Allow users to add more movies before calculations.

Lab 3 — Intelligent Agents

1. Simple Reflex Agents

These agents select actions based on the **current percept** only. They do not store history.

Example: Thermostat — monitors room temperature and turns heater on/off based on a set point.

2. Model-Based Reflex Agents

These agents maintain an **internal state** to track aspects of the environment that may change or be hidden.

Example: Self-Driving Car — uses sensors and maintains internal state (speed, location of vehicles, traffic signals).

3. Goal-Based Agents

Act to achieve **specific goals** using the current state and desired outcome.

Example: Robot Vacuum Cleaner — maps the room, plans a path, avoids obstacles to clean the entire floor.

4. Utility-Based Agents

Maximize a **utility function** (measure of satisfaction). They evaluate actions to pick the one with maximum utility.

Example: Investment AI — trades stocks to maximize expected returns.

5. Learning Agents

Learn from environment, experiences, and feedback to **improve performance over time**.

Example: Chatbot — improves responses based on user feedback.

6. Multi-Agent Systems

Multiple agents interact or collaborate to solve complex problems.

Example: Autonomous Drone Fleet — search and rescue operations with coordinated coverage.

Scenario: Smart Home Temperature Control

A smart home with multiple rooms, each with a thermostat. The goal is to maintain **22°C** in each room.

Simple Reflex Agent — Python Implementation

```
class SimpleReflexAgent:
    def __init__(self, desired_temperature):
        self.desired_temperature = desired_temperature

    def perceive(self, current_temperature):
        return current_temperature

    def act(self, current_temperature):
        if current_temperature < self.desired_temperature:
            action = "Turn on heater"
        else:
            action = "Turn off heater"
        return action

# Simulating different rooms
rooms = {
    "Living Room": 18,
    "Bedroom": 22,
    "Kitchen": 20,
    "Bathroom": 24
}

desired_temperature = 22
agent = SimpleReflexAgent(desired_temperature)

for room, temperature in rooms.items():
    action = agent.act(temperature)
    print(f"{room}: Current temperature = {temperature}°C. {action}.")
```

Output:

```
Living Room: Current temperature = 18°C. Turn on heater.
Bedroom: Current temperature = 22°C. Turn off heater.
Kitchen: Current temperature = 20°C. Turn on heater.
Bathroom: Current temperature = 24°C. Turn off heater.
```

Lab 3 Task: Model-Based Reflex Agent

This agent not only checks the current temperature but also **remembers the previous action** to avoid turning the heater on or off unnecessarily.

```
class ModelBasedReflexAgent:
    def __init__(self, desired_temperature):
        self.desired_temperature = desired_temperature
        self.last_action = None  # Internal state

    def act(self, current_temperature):
        if current_temperature < self.desired_temperature:
            if self.last_action != "Turn on heater":
                action = "Turn on heater"
            else:
                action = "Heater already on"
        else:
            if self.last_action != "Turn off heater":
                action = "Turn off heater"
            else:
                action = "Heater already off"
        self.last_action = action
        return action

# Test the model-based agent
agent2 = ModelBasedReflexAgent(22)
print(agent2.act(18))  # Turn on heater
print(agent2.act(18))  # Heater already on
print(agent2.act(24))  # Turn off heater
print(agent2.act(24))  # Heater already off
```

Lab 4 — Graph in Python (Tic Tac Toe & LUHN Algorithm)

Classic Game: Tic Tac Toe

Two players take turns marking a 3×3 grid with "X" or "O". The game continues until a player forms a straight line or all squares are filled (draw).

```
square = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

def main():
    player = 1
    status = -1
    while status == -1:
        board()
        if player % 2 == 1:
            player = 1
        else:
            player = 2
        print('\nPlayer', player)
        choice = int(input('Enter a number: '))
        if player == 1:
            mark = 'X'
        else:
            mark = 'O'

        if choice == 1 and square[1] == 1:
            square[1] = mark
        elif choice == 2 and square[2] == 2:
            square[2] = mark
        elif choice == 3 and square[3] == 3:
            square[3] = mark
        elif choice == 4 and square[4] == 4:
            square[4] = mark
        elif choice == 5 and square[5] == 5:
            square[5] = mark
        elif choice == 6 and square[6] == 6:
            square[6] = mark
        elif choice == 7 and square[7] == 7:
            square[7] = mark
        elif choice == 8 and square[8] == 8:
            square[8] = mark
        elif choice == 9 and square[9] == 9:
            square[9] = mark
        else:
            print('Invalid move')
            player -= 1

        status = game_status()
        player += 1

    print('RESULT')
    if status == 1:
        print('Player', player - 1, 'win')
    else:
        print('Game draw')

def game_status():
    if square[1] == square[2] and square[2] == square[3]:
        return 1
    elif square[4] == square[5] and square[5] == square[6]:
        return 1
    elif square[7] == square[8] and square[8] == square[9]:
        return 1
    elif square[1] == square[4] and square[4] == square[7]:
        return 1
    elif square[2] == square[5] and square[5] == square[8]:
        return 1
    elif square[3] == square[6] and square[6] == square[9]:
        return 1
    elif square[1] == square[5] and square[5] == square[9]:
        return 1
    elif square[3] == square[5] and square[5] == square[7]:
```

```
        return 1
    elif square[1] != 1 and square[2] != 2 and square[3] != 3 and \
         square[4] != 4 and square[5] != 5 and square[6] != 6 and \
         square[7] != 7 and square[8] != 8 and square[9] != 9:
        return 0
    else:
        return -1

def board():
    print('\n\n\tTic Tac Toe\n\n')
    print('Player 1 (X) - Player 2 (O)')
    print()
    print('     |     |     ')
    print(' ', square[1], ' | ', square[2], ' | ', square[3])
    print('_____|_____|_____')
    print('     |     |     ')
    print(' ', square[4], ' | ', square[5], ' | ', square[6])
    print('_____|_____|_____')
    print('     |     |     ')
    print(' ', square[7], ' | ', square[8], ' | ', square[9])
    print('     |     |     ')

main()
```

The LUHN Algorithm (Credit Card Validation)

The Luhn algorithm is used to verify credit/debit card numbers and SIM card serial numbers.

Steps:

1. Remove the rightmost digit (checking digit).
2. Reverse the remaining digits.
3. Double digits at even indices (0, 2, 4, ...). If result > 9, subtract 9.
4. Sum all results plus the checking digit.
5. If the total is divisible by 10, the number is **valid**.

Example with valid MasterCard: `5 8 9 3 8 0 4 1 1 5 4 5 7 2 8 9`

Step	Result
Starting number	5 8 9 3 8 0 4 1 1 5 4 5 7 2 8 9
Remove last digit (X=9)	5 8 9 3 8 0 4 1 1 5 4 5 7 2 8 X
Reverse remaining	8 2 7 5 4 5 1 1 4 0 8 3 9 8 5 X
Double even indices	16 2 14 5 8 5 2 1 8 0 16 3 18 8 10 X
Subtract 9 if >9	7 2 5 5 8 5 2 1 8 0 7 3 9 8 1 X

Sum = 7+2+5+5+8+5+2+1+8+0+7+3+9+8+1+**9** = **80** (divisible by 10) → **Valid!**

Lab 4 Tasks

1. **Code for LUHN Algorithm** in Python.
2. Write a Python program to **remove punctuation** from a given string.
3. Write a Python program to **sort a sentence in alphabetical order**.

Lab 5 — Uninformed Search (BFS & DFS)

1. Breadth-First Search (BFS)

```
import collections

def bfs(graph, root):
    visited, queue = set(), collections.deque([root])
    visited.add(root)
    while queue:
        vertex = queue.popleft()
        print(str(vertex) + " ", end="")
        for neighbour in graph[vertex]:
            if neighbour not in visited:
                visited.add(neighbour)
                queue.append(neighbour)

if __name__ == '__main__':
    graph = {0: [1, 2], 1: [2], 2: [3], 3: [1, 2]}
    print("Following is Breadth First Traversal: ")
    bfs(graph, 0)
```

Output: Following is Breadth First Traversal: 0 1 2 3

2. Depth-First Search (DFS)

```
def dfs(graph, start, visited=None):
    if visited is None:
        visited = set()
    visited.add(start)
    print(start)
    for next_node in graph[start] - visited:
        dfs(graph, next_node, visited)
    return visited

graph = {'0': set(['1', '2']),
         '1': set(['0', '3', '4']),
         '2': set(['0']),
         '3': set(['1']),
         '4': set(['2', '3'])}
dfs(graph, '0')
```

Lab 5 Tasks

1. **DFS with Stack & Node** — Implement DFS iteratively using a stack.
2. **Research:** "Inorder, Preorder, Postorder" tree traversals and implement them in DFS style.

Lab 6 — Searching Algorithms: Informed / Heuristic Search (DLS)

Depth-Limited Search (DLS)

```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': ['G', 'H'],
    'E': [],
    'F': ['I', 'K'],
    'G': [],
    'H': ['L'],
    'I': [],
    'K': ['M'],
    'L': [],
    'M': []
}

def dls(start, goal, path, level, max_depth):
    print(level)
    print(start)
    path.append(start)
    if start == goal:
        return path
    if level == max_depth:
        return path
    for child in graph[start]:
        if dls(child, goal, path, level + 1, max_depth):
            return path
    path.pop()
    return False

start = 'A'
goal = input("Enter the goal state: ")
max_depth = int(input("Enter the limit: "))
path = list()
res = dls(start, goal, path, 0, max_depth)
if res:
    print(path)
else:
    print("Goal not found within depth limit.")
```

Lab 6 Tasks

1. **BFS without Queue & without Node** — Implement BFS using only lists.
2. **BFS with Queue & Node** — Standard BFS implementation with proper queue and node structure.

Lab 7 — Game Search (A* Algorithm)

A* Search Algorithm

A* combines the actual cost from the start node (**g(n)**) and a heuristic estimate to the goal (**h(n)**) to find the optimal path.

```
from collections import deque

class Graph:
    def __init__(self, adjacency_list):
        self.adjacency_list = adjacency_list

    def get_neighbors(self, v):
        return self.adjacency_list[v]

    def h(self, n):
        H = {'A': 1, 'B': 1, 'C': 1, 'D': 1}
        return H[n]

    def a_star_algorithm(self, start_node, stop_node):
        open_list = set([start_node])
        closed_list = set([])
        g = {}
        g[start_node] = 0
        parents = {}
        parents[start_node] = start_node

        while len(open_list) > 0:
            n = None
            for v in open_list:
                if n == None or g[v] + self.h(v) < g[n] + self.h(n):
                    n = v

            if n == None:
                print('Path does not exist!')
                return None

            if n == stop_node:
                reconst_path = []
                while parents[n] != n:
                    reconst_path.append(n)
                    n = parents[n]
                reconst_path.append(start_node)
                reconst_path.reverse()
                print('Path found: {}'.format(reconst_path))
                return reconst_path

            for (m, weight) in self.get_neighbors(n):
                if m not in open_list and m not in closed_list:
                    open_list.add(m)
                    parents[m] = n
                    g[m] = g[n] + weight
                else:
                    if g[m] > g[n] + weight:
                        g[m] = g[n] + weight
                        parents[m] = n
                        if m in closed_list:
                            closed_list.remove(m)
                            open_list.add(m)

            open_list.remove(n)
            closed_list.add(n)

        print('Path does not exist!')
        return None

adjacency_list = {
    'A': [('B', 1), ('C', 3), ('D', 7)],
    'B': [('D', 5)],
    'C': [('D', 12)]
}
graph1 = Graph(adjacency_list)
graph1.a_star_algorithm('A', 'D')
```

Output: Path found: ['A', 'B', 'D']

Lab 7 Task

1. **Write code for A* Algorithm** with a custom heuristic and a more complex graph. Test with different start/goal nodes and verify optimal path selection.

Lab 8 — Machine Learning (File Read, Data Preprocessing)

Topics covered:

- Reading CSV/data files using `pandas.read_csv()`
- Handling missing values (drop, fill, impute)
- Feature scaling (normalization, standardization)
- Encoding categorical variables (one-hot encoding, label encoding)
- Train-test split using `sklearn.model_selection.train_test_split`

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder

df = pd.read_csv('data.csv')
df.fillna(df.mean(), inplace=True)
X = df.drop('target', axis=1)
y = df['target']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Lab 9 — Machine Learning (Naïve Bayes, D-Tree, KNN, SVM)

Implementation of classifiers:

```
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC

# Naïve Bayes
nb = GaussianNB()
nb.fit(X_train, y_train)

# Decision Tree
dt = DecisionTreeClassifier()
dt.fit(X_train, y_train)

# KNN
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)

# SVM
svm = SVC(kernel='rbf')
svm.fit(X_train, y_train)
```

Lab 10 — Introduction to Deep Learning

Topics: Neural networks, activation functions (ReLU, sigmoid, softmax), forward/backward propagation, using TensorFlow/Keras.

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

model = Sequential([
    Dense(64, activation='relu', input_shape=(input_dim,)),
    Dense(32, activation='relu'),
    Dense(1, activation='sigmoid')
])
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
model.fit(X_train, y_train, epochs=10, batch_size=32)
```

Lab 11 — Data Visualization using Python

Using `matplotlib` and `seaborn` for plotting:

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(10, 6))
sns.scatterplot(x='feature1', y='feature2', hue='target', data=df)
plt.title('Feature Scatter Plot')
plt.show()

sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.show()
```

Lab 12 — Precision, Accuracy, Recall

Evaluation metrics for classification models:

Metric	Formula	Description
Accuracy	(TP + TN) / Total	Overall correctness
Precision	TP / (TP + FP)	How many predicted positives are actual positives
Recall	TP / (TP + FN)	How many actual positives were predicted correctly
F1-Score	2 × (P×R)/(P+R)	Harmonic mean of precision and recall

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("F1 Score:", f1_score(y_test, y_pred))
```

Lab 13 — Project

Students will develop a complete AI/ML project incorporating concepts from all previous labs. The project should include:

- Problem definition and dataset selection
- Data preprocessing and exploratory data analysis
- Model selection and training
- Hyperparameter tuning
- Evaluation using appropriate metrics
- Visualization of results

Suggested Project Ideas: Spam email classifier, sentiment analysis, image classification, stock price prediction, chatbot development.

Lab 14 — Project Evaluation

Final evaluation of the project including:

- Code review and documentation quality
- Model performance metrics
- Presentation and demonstration
- Viva voce / Q&A session
- Report submission

Student Name: *Asad*

Roll Number: *141* | ***Class:*** *BSAI 4C*

Instructor: *Rasikh Ali*

Faculty of Software Engineering — Superior University Lahore, Pakistan

— End of Lab Manual —